Chance-Constrained NMPC using Covariance-based Signed-Distance Tightening

AEROSP 740 MPC W26
Kristian Popov

April 26, 2026

Abstract **– Real-world safety-critical aerospace and robotic systems are required to reason about uncertainty due to noise in the true dynamics. In this paper, we develop a chance-constrained nonlinear model predictive control (NMPC) framework for obstacle avoidance in three-dimensional environments with general obstacle geometry. Our method represents obstacles using signed distance fields (SDFs) and enforces probabilistic collision avoidance by projecting the marginalized-predicted state covariance onto local obstacle normal directions. This converts stochastic safety constraints into deterministic tightening margins that can be incorporated efficiently within the NMPC optimization problem. Simulation testing in multiple cluttered environments using a nonlinear quadrotor model, perturbed with Gaussian process noise, demonstrates that our method is capable of generating dynamically feasible trajectories that satisfy actuator and workspace constraints while avoiding collision with obstacles. These results suggest that our covariance-based SDF tightening provides a scalable and practical approach to stochastic collision avoidance for nonlinear MPC.**

1 Motivation

Safe autonomous navigation in real-world aerospace and robotic systems requires moving from an initial state to a desired goal while avoiding collisions with environmental obstacles under uncertain dynamics. This problem is fundamentally challenging for two reasons. First, the true system dynamics are rarely known exactly and are often affected by stochastic process disturbances, modeling error, and actuator noise among other factors. Second, collision-avoidance constraints are typically nonconvex because obstacles induce disconnected feasible regions, narrow passages, and complex geometry. The combination of nonlinear dynamics, uncertainty, and nonconvex safety constraints makes real-time planning particularly difficult.

Model predictive control (MPC) provides a natural framework for this setting because it repeatedly solves a finite-horizon optimal control problem using the current state estimate and updated predictions. In uncertain environments, several stochastic MPC formulations have been developed, including tube-based MPC, scenario MPC, sampling-based chance-constrained MPC, and distributionally robust MPC [7, 5, 8]. Scenario and particle-based methods can provide flexible uncertainty representations, but their computational burden often scales poorly with horizon length, system dimension, or required confidence level, making them challenging for fast receding-horizon implementation.

For this reason, we follow a complementary line of work that uses parametric uncertainty representations, most commonly Gaussian state predictions propagated through local linearizations of the dynamics [2, 1]. Such methods replace large particle sets with means and covariances, significantly reducing online computational cost while retaining useful probabilistic information.

To address geometric collision avoidance under uncertainty, we represent obstacles using signed distance fields (SDFs). The SDF provides both the local distance-to-obstacle value and its gradient direction. By linearizing the SDF near a reference trajectory and projecting the predicted covariance onto the local obstacle normal, probabilistic collision constraints can be converted into deterministic tightening margins. This yields a computationally efficient approximation of stochastic obstacle avoidance that remains compatible with nonlinear MPC and general obstacle geometry.

2 Problem Setting

This section introduces the quadrotor dynamics, stochastic disturbance model, and covariance-based chance-constrained SDF obstacle formulation.

2.1 Dynamical Quadrotor System

Consider a dynamical quadrotor system with state $x = [p_x, p_y, p_z, v_x, v_y, v_z, \phi, \theta, \psi, \omega_x, \omega_y, \omega_z]^\top$, where $[p_x, p_y, p_z]^\top$ denotes the quadrotor's pose p, $v = [v_x, v_y, v_z]^\top$ denotes its velocity, $\eta = [\phi, \theta, \psi]^\top$ are the Euler angles, and $\omega = [\omega_x, \omega_y, \omega_z]^\top$ denotes the angular velocities. We consider the action vector $u = [T, \tau_x, \tau_y, \tau_z]^\top$, where T is the thrust and $\tau = [\tau_x, \tau_y, \tau_z]^\top$ denotes body-frame torques. The full continuous dynamics of the quadrotor system are given by

$$\dot{p} = v \tag{1}$$

$$\dot{v} = \frac{T}{m} R(\phi, \theta, \psi) e_3 - g e_3 \tag{2}$$

$$\dot{\eta} = E(\phi, \theta)\omega \tag{3}$$

$$\dot{\omega} = J^{-1}(\tau - \omega \times J\omega) \tag{4}$$

where m is the mass of the system, g is the acceleration due to gravity, $R(\phi, \theta, \psi)$ is the 3-2-1 rotation matrix given by

$$R(\phi, \theta, \psi) = R_z(\psi) R_y(\theta) R_x(\phi),$$

$$R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}, \qquad R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi & \cos\phi \end{bmatrix},$$

J is the inertia matrix

$$J = \begin{bmatrix} J_x & 0 & 0 \\ 0 & J_y & 0 \\ 0 & 0 & J_z \end{bmatrix},$$

and $E(\phi, \theta)$ is the Euler-rate mapping matrix

$$E = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \dfrac{\sin\phi}{\cos\theta} & \dfrac{\cos\phi}{\cos\theta} \end{bmatrix}.$$

This model is adapted from the quadrotor model given in [4].

We model the quadrotor as a stochastic dynamical system subject to additive Gaussian process noise. After discretization with sampling time Δt, the system evolves according to

$$x_{k+1} = f_d(x_k, u_k) + D w_k, \qquad w_k \sim \mathcal{N}(0, W),$$

where f_d is the discretized quadrotor dynamics, w_k is a zero-mean Gaussian disturbance, W is the disturbance covariance, and D maps the disturbance to the state. In this work, we consider disturbances acting on the translational acceleration/velocity dynamics.

2.2 Chance-Constrained SDF Tightening

Let $d_i(p)$ denote the signed distance from position p to obstacle i $\mathcal{O}_i$. We follow the procedure of [3] and enforce safety by requiring

$$d_i(p_k) \geq r_{\text{robot}}, \quad \forall i, k.$$

Let $\nabla d_i(p)$ denote the local SDF gradient. Around a reference point $p_{\text{ref},k}$, we linearize the SDF as

$$d_i(p_k) \approx d_i(p_{\text{ref},k}) + \nabla d_i(p_{\text{ref},k})^\top (p_k - p_{\text{ref},k}).$$

Assuming the position uncertainty at time k is approximately Gaussian,

$$p_k \sim \mathcal{N}(\bar{p}_k, \Sigma_{p,k}),$$

the scalar linearized SDF is also approximately Gaussian with standard deviation

$$\sigma_{d,i,k} = \sqrt{\nabla d_i(p_{\text{ref},k})^\top \Sigma_{p,k} \nabla d_i(p_{\text{ref},k})}.$$

Therefore, the chance constraint

$$\mathbb{P}\left(d_i(p_k) \geq r_{\text{robot}}\right) \geq 1 - \alpha$$

is approximated by the deterministic tightened constraint

$$d_i(p_{\text{ref},k}) + \nabla d_i(p_{\text{ref},k})^\top \left(p_k - p_{\text{ref},k}\right) \geq r_{\text{robot}} + \beta \sigma_{d,i,k},$$

where

$$\beta = \Phi^{-1}(1 - \alpha),$$

where Φ is the cumulative distribution function of the standard normal distribution, hence, $\beta = \Phi^{-1}(1-\alpha)$ is the one-sided Gaussian quantile associated with the user-specified threshold α.

Using a point-based or polygonal representation of the environment, the obstacle geometry is encoded through an SDF oracle

$$(d_i(p), \nabla d_i(p)) = \text{SDF}(p, \mathcal{O}_i),$$

where $\mathcal{O}_i$ is specified by a set of 3D points defining a polygon, mesh, wall, or surface patch. The SDF oracle returns both the distance-to-obstacle value and the local outward gradient direction. The MPC then enforces the linearized chance-tightened obstacle constraint

$$d_i(p_{\text{ref},k}) + \nabla d_i(p_{\text{ref},k})^\top \left(p_k - p_{\text{ref},k}\right) \geq r_{\text{robot}} + \Phi^{-1}(1-\alpha)\sqrt{\nabla d_i(p_{\text{ref},k})^\top \Sigma_{p,k} \nabla d_i(p_{\text{ref},k})}.$$

2.3 Covariance Propagation

We assume

$$w_k \sim \mathcal{N}(0, W)$$

with the injection matrix

$$D = \begin{bmatrix} 0_{3\times 3} \\ I_3 \\ 0_{6\times 3} \end{bmatrix}.$$

Along the trajectory, we will linearize our dynamics

$$A_k = \frac{\partial f_d}{\partial x}(x_k, u_k), \qquad B_k = \frac{\partial f_d}{\partial u}(x_k, u_k). \tag{5}$$

A stabilizing feedback gain K is obtained using LQR. We approximate the closed-loop error dynamics as

$$e_{k+1} = (A_k + B_k K) e_k + D w_k. \tag{6}$$

Therefore, as shown in [6], the covariance evolves according to

$$\Sigma_{k+1} = (A_k + B_k K)\Sigma_k (A_k + B_k K)^\top + D W D^\top \tag{7}$$

with $\Sigma_0 = 0$. We will use the Σ_k in the obstacle-tightening constraints.

3 MPC Problem and Formulation

We consider the problem of safely commanding the quadrotor from an initial state x_0 to a goal state $x_{\mathcal{G}}$ through a cluttered environment. Given the stochastic quadrotor model and chance-constrained obstacle formulation introduced in Section 2, we construct a receding-horizon nonlinear model predictive controller. We consider a cost where the stage cost penalizes deviation from the goal state and excessive control effort, while a terminal cost promotes convergence near the horizon endpoint.

$$\min_{\{x_k,u_k\}_{k=0}^{N}} \quad \sum_{k=0}^{N-1} \big((x_k - x_{\mathcal{G}})^\top Q(x_k - x_{\mathcal{G}}) + (u_k - u_{\text{hover}})^\top R(u_k - u_{\text{hover}})\big) + (x_N - x_{\mathcal{G}})^\top Q_f(x_N - x_{\mathcal{G}}) \quad (8)$$

$$\begin{aligned}
\text{subject to} \quad & x_0 = x_{\text{current}} && (9)\\
& x_{k+1} = f_d(x_k, u_k), \quad k = 0, \dots, N-1 && (10)\\
& p_{\min} \le p_k \le p_{\max} && (11)\\
& -\theta_{\max} \le [\phi_k, \theta_k, \psi_k]^\top \le \theta_{\max} && (12)\\
& T_{\min} \le T_k \le T_{\max} && (13)\\
& -\tau_{\max} \le \tau_k \le \tau_{\max} && (14)\\
& d_i(p_{\text{ref},k}) + \nabla d_i(p_{\text{ref},k})^\top (p_k - p_{\text{ref},k}) \ge r_{\text{robot}} + \beta\sqrt{\nabla d_i(p_{\text{ref},k})^\top \Sigma_{p,k} \nabla d_i(p_{\text{ref},k})}, && (15)\\
& \qquad \text{for } i = 1, \dots, N_{\text{obs}}, \quad k = 0, \dots, N.
\end{aligned}$$

3.1 Implementation

In this section, we will explore how we use Algorithm 1 to solve the optimization problem posed above.

Algorithm 1: Sequentially Tightened Chance-Constrained NMPC

```
for each control step do
    Measure current state x_t;
    Initialize trajectory using shifted previous solution;
    for j = 1, ..., N_scp do
        Linearize dynamics and obstacle constraints about current reference;
        Propagate covariance along the horizon;
        Update deterministic tightening margins;
        Solve the resulting nonlinear program;
    end
    Apply first control input u_0^⋆;
    Shift horizon and repeat;
end
```

3.1.1 Numerical Implementaiton

We define the state X and action U vectors as

$$X = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_N \end{bmatrix}, \qquad U = \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}. \quad (16)$$

We also define the decision vector z as

$$z = \begin{bmatrix} X \\ U \end{bmatrix} \quad (17)$$

The finite-horizon control problem is transcribed into a nonlinear program using a direct multiple-shooting formulation. Each predicted state x_k is treated as an optimization variable, while the nonlinear dynamics are enforced through equality constraints

$$x_{k+1} - f_d(x_k, u_k) = 0, \qquad k = 0, \dots, N-1. \tag{18}$$

The discrete-time dynamics function f_d is obtained from the continuous quadrotor dynamics using a fourth-order Runge–Kutta (RK4) integrator. In our implementation, we inject Gaussian noise into the translational velocity channels after each discrete integration step, corresponding to random acceleration disturbances over one sampling interval.

We implement the nonlinear program in CasADi, which provides automatic differentiation of the dynamics and constraints. IPOPT is used as the resulting nonlinear programming solver through a primal-dual interior-point method.

Let $\bar{X}^{(j)} = \{\bar{x}_k^{(j)}\}_{k=0}^N$ denote the nominal trajectory at refinement iteration j. The covariance recursion and SDF linearizations are computed about $\bar{X}^{(j)}$. After solving the tightened NLP, the resulting optimizer becomes the next reference trajectory $\bar{X}^{(j+1)}$.

Because the obstacle constraints depend on the local geometry and predicted uncertainty, the chance-tightening margins are updated iteratively at each control step. Given a reference trajectory $\{\bar{p}_k\}_{k=0}^N$ (the poses of $\{\bar{x}_k^{(j)}\}_{k=0}^N$), the covariance sequence Σ_k is first propagated along the horizon using the linearized closed-loop dynamics. For each obstacle i and stage k, the local signed-distance gradient $\nabla d_i(\bar{p}_k)$ is then evaluated and used to compute the projected uncertainty

$$\sigma_{i,k} = \sqrt{\nabla d_i(\bar{p}_k)^\top \Sigma_{p,k} \nabla d_i(\bar{p}_k)}.$$

The corresponding deterministic tightening margin is $\delta_{i,k} = \beta \sigma_{i,k}$. The nonlinear program is then solved subject to the tightened linearized obstacle constraints

$$d_i(\bar{p}_k) + \nabla d_i(\bar{p}_k)^\top (p_k - \bar{p}_k) \geq r_{\text{robot}} + \delta_{i,k}.$$

The covariance sequence is held fixed during each NLP solve and updated in an outer sequential refinement loop. In our implementation, two such refinement iterations are performed per MPC step. After convergence, only the first control input $u_0^\star$ is applied, the horizon is shifted forward, and the previous solution is used to warm-start the next optimization.

The feedback gain K used for covariance propagation and nominal trajectory tracking is obtained from a locally linearized hover model. More specifically, the nonlinear quadrotor dynamics are linearized near the equilibrium

$$x = x_{\text{hover}}, \qquad u = u_{\text{hover}} = \begin{bmatrix} mg & 0 & 0 & 0 \end{bmatrix}^\top, \tag{19}$$

and discretized to produce (A_d, B_d). The gain is then computed using the discrete-time LQR problem

$$K = -\text{dlqr}(A_d, B_d, Q, R). \tag{20}$$

This gain stabilizes small deviations around hover and provides a computationally inexpensive local feedback law for tracking the optimized nominal trajectory, and the control applied to the plant is

$$u_k = u_k^\star + K(x_k - \bar{x}_k),$$

where $u_k^\star$ is the NMPC nominal input.

3.1.2 SDF

Obstacle avoidance is handled through a signed distance field (SDF) representation of the environment. For each obstacle $\mathcal{O}_i$, the SDF oracle returns

$$(d_i(p), \nabla d_i(p)), \tag{21}$$

where $d_i(p)$ is the signed distance from the quadrotor position p to the obstacle boundary, and $\nabla d_i(p)$ is the outward distance gradient.

The environment may be represented using polygonal meshes, planar surfaces, or point-defined geometry. For polygonal obstacles, the signed distance is computed by projecting the query point onto the obstacle plane and checking whether the projected point lies inside the polygon. If so, the perpendicular distance to the plane is used; otherwise, the nearest edge or vertex distance is used.

At each MPC iteration, the SDF is evaluated along the reference trajectory. These values are then linearized to produce differentiable obstacle constraints suitable for nonlinear optimization. This representation enables the controller to handle general obstacle geometries without requiring spherical approximations or sampling-based collision checking.

3.2 Theoretical Analysis

The proposed controller provides local guarantees of constraint-enforcement through a deterministic approximation of stochastic chance-constraints based on the Gaussian uncertainty propagation. Under additive Gaussian process disturbances

$$w_k \sim \mathcal{N}(0, W), \tag{22}$$

and local linearization of the closed-loop dynamics about the nominal trajectory, the tracking error is

$$e_{k+1} = (A_k + B_k K)e_k + Dw_k. \tag{23}$$

If e_0 is Gaussian, then e_k remains Gaussian under the linearized model, so covariance satisfies

$$\Sigma_{k+1} = (A_k + B_k K)\Sigma_k(A_k + B_k K)^\top + DWD^\top. \tag{24}$$

Let p_k denote the predicted position state and let $d_i(p_k)$ be the signed distance to obstacle i. Linearizing the signed distance field about a reference position $p_{\text{ref},k}$ gives

$$d_i(p_k) \approx d_i(p_{\text{ref},k}) + \nabla d_i(p_{\text{ref},k})^\top (p_k - p_{\text{ref},k}). \tag{25}$$

Since affine transformations of Gaussian random variables remain Gaussian, the linearized signed distance is normally distributed with variance

$$\sigma_{d,i,k}^2 = \nabla d_i^\top \Sigma_{p,k} \nabla d_i. \tag{26}$$

Therefore, the chance constraint

$$\mathbb{P}\big(d_i(p_k) \geq r_{\text{robot}}\big) \geq 1 - \alpha \tag{27}$$

is equivalently enforced for the linearized model by the deterministic tightening condition

$$d_i(p_{\text{ref},k}) + \nabla d_i^\top (p_k - p_{\text{ref},k}) \geq r_{\text{robot}} + \Phi^{-1}(1-\alpha)\sigma_{d,i,k}. \tag{28}$$

Therefore, if we assume that the linearized dynamics and first-order SDF approximation are valid, we are able to provide a local probabilistic guarantees on the obstacle-constraint satisfaction.

4 Experimental Results

We consider the quadrotor system described in Section 2, with mass $m = 1.0$ kg, $g = 9.81$ m/s^2, $J_x = 0.02$, $J_y = 0.02$, $J_z = 0.04$. Our MPC controller uses a horizon $N = 20$, a sampling rate $T_s = 0.10$ s, and we have a maximum simulation length of $T_{\text{sim}} = 80$ s. To improve convergence of the nonlinear obstacle-constraints, each control step performs $N_{\text{SCP}} = 2$ successive trajectory refinements using updated covariance and SDF linearizations before applying the first control action.

We also define the nominal hover input as

$$u_{\text{hover}} = [mg, 0, 0, 0]^\top.$$

Our system has thrust (T) input constraints

$$0.2mg \leq T \leq 1.8mg,$$

and torque (τ) limits

$$|\tau_x| \leq 0.25, \qquad |\tau_y| \leq 0.25, \qquad |\tau_z| \leq 0.15.$$

Euler-angle limits are constrained to

$$|\phi|, |\theta|, |\psi| \leq 35^\circ.$$

The quadrotor safety radius used in all obstacle constraints is

$$r_{\text{robot}} = 0.25 \text{ m}.$$

We assume a Gaussian process noise with standard deviations

$$\sigma = \left[0.25, 0.25, 0.18\right]^\top,$$

with covariance matrix

$$W = \text{diag}(\sigma^2).$$

We define our acceptable chance-constraint failure rate to $\alpha = 0.05$, corresponding to a one-sided Gaussian tightening coefficient

$$\beta = \Phi^{-1}(0.95) \approx 1.645.$$

Our cost function applies the same quadratic penalties in all three experiments:

$$Q = \text{diag}(30, 30, 35, 4, 4, 5, 2, 2, 2, 0.2, 0.2, 0.2),$$

$$Q_f = \text{diag}(80, 80, 100, 8, 8, 10, 4, 4, 4, 0.5, 0.5, 0.5),$$

$$R = \text{diag}(0.08, 0.5, 0.5, 0.5).$$

Our simulator's stopping (success) criteria is that $\|p - p_{\mathcal{G}}\| < 0.25$. The three scenarios that we tested our method on are formalized below, and Figures 1 to 3 show the executed trajectories from our MPC-scheme and the evolution of the corresponding control inputs over time.

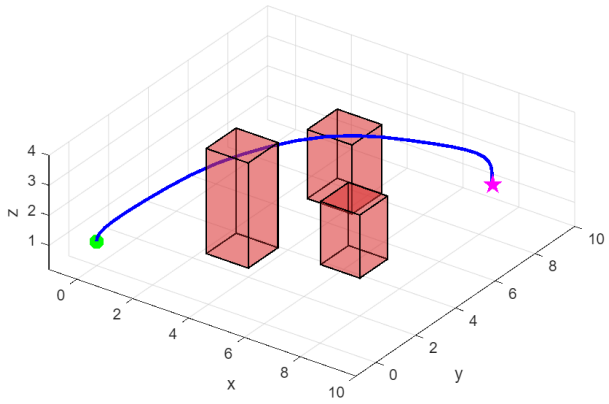
Case 1:

Initial State: $x_0 = (0, 0, 1.0, 0_{1\times 9})^\top$ Goal State: $x_{\mathcal{G}} = (8.5, 8.0, 2.0, 0_{1\times 9})^\top$

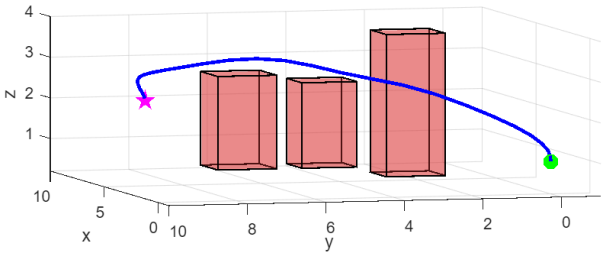
Workspace Bounds: [-1.0,-1.0,0.2]$^\top \leq p \leq$ [10.0,10.0,4.0]$^\top$

Obstacle Constraints:

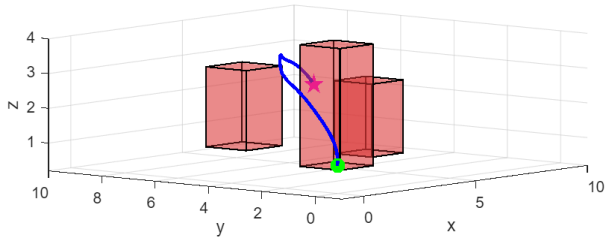
1. $x \in [2.5, 4.0], \quad y \in [2.0, 3.5], \quad z \in [0.5, 4.0]$
2. $x \in [5.6, 7.0], \quad y \in [3.4, 4.8], \quad z \in [0.5, 2.6]$
3. $x \in [3.2, 4.8], \quad y \in [6.1, 7.6], \quad z \in [0.7, 3.0]$



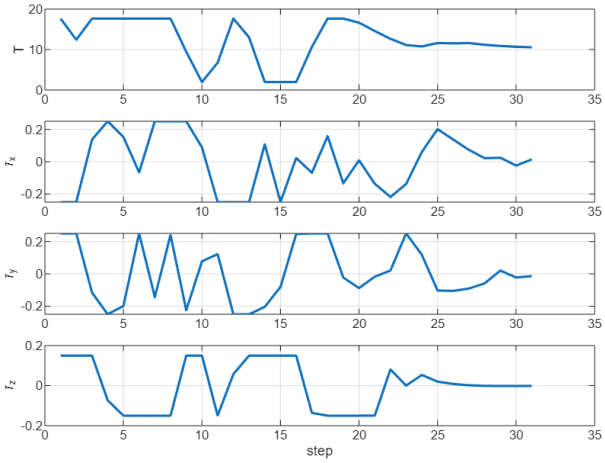
(a) Case 1 trajectory.



(b) Case 1 trajectory.



(c) Case 1 trajectory.



(d) Case 1 control input evolution.

Figure 1: Figures a-c show a rendering of the quadrotor trajectory through the Case 1 environment. The initial pose is represented using a green dot, the goal pose is shown as a pink star, the trajectory of the quadrotor is given as a blue line, and the polyhedral obstacles are rendered as translucent red boxes. Figure d shows the evolution of the control inputs over time.

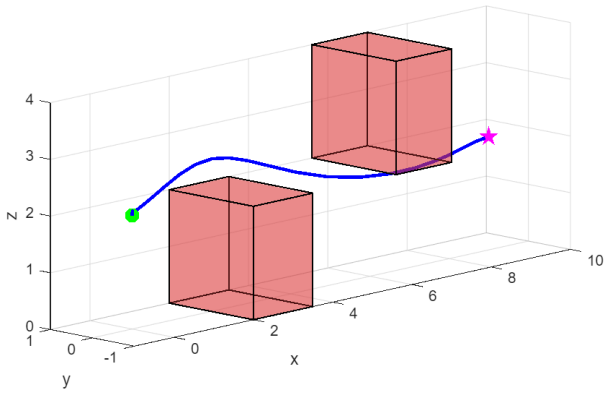
Case 2:

$$\text{Initial State: } x_0 = (0, 0, 2.0, 0_{1\times 9})^\top \qquad \text{Goal State: } x_\mathcal{G} = (9.0, 0, 2.0, 0_{1\times 9})^\top$$

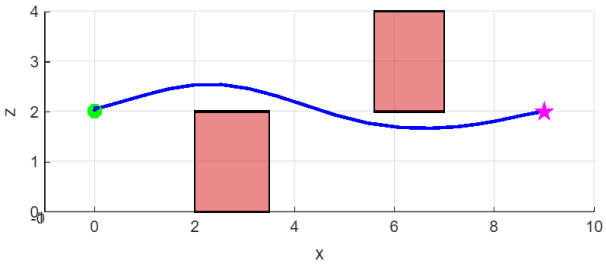
Workspace Bounds: $[-1.0, -1.0, 0.0]^\top \leq p \leq [10.0, 1.0, 4.0]^\top$

Obstacle Constraints:

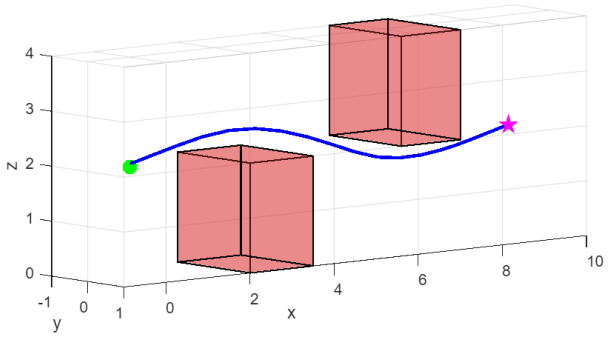
1. $x \in [2.0, 3.5], \quad y \in [-1.0, 1.0], \quad z \in [0.0, 2.0]$
2. $x \in [5.6, 7.0], \quad y \in [-1.0, 1.0], \quad z \in [2.0, 4.0]$



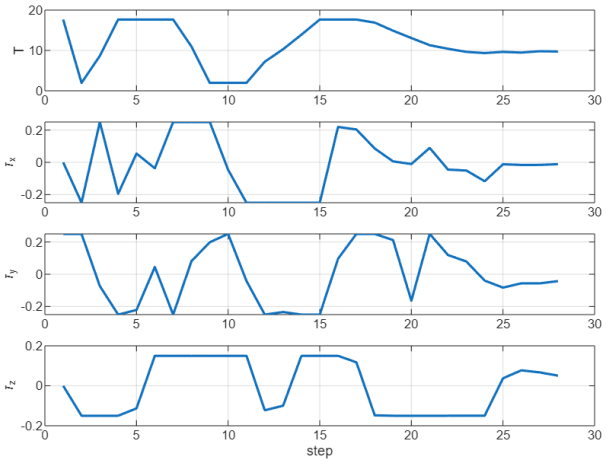
(a) Case 2 trajectory.



(b) Case 2 trajectory.



(c) Case 2 trajectory.



(d) Case 2 trajectory.

Figure 2: Figures a-c show a rendering of the quadrotor trajectory through the Case 2 environment. The initial pose is represented using a green dot, the goal pose is shown as a pink star, the trajectory of the quadrotor is given as a blue line, and the polyhedral obstacles are rendered as translucent red boxes. Figure d shows the evolution of the control inputs over time.

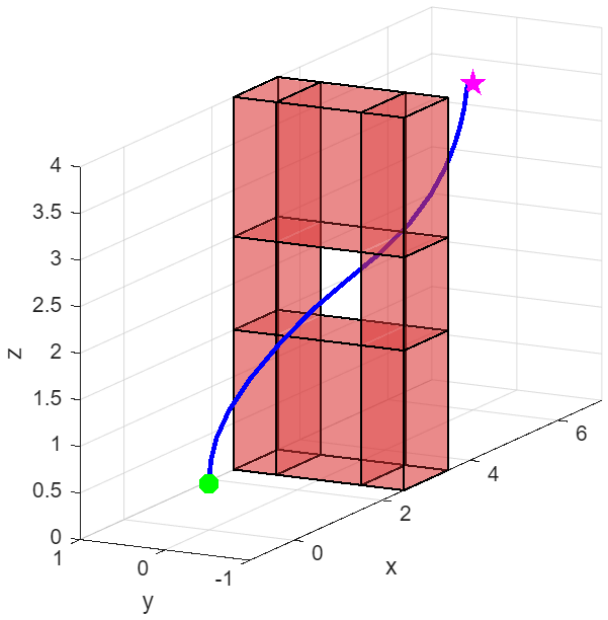
Case 3:

Initial State: $x_0 = (0, 0, 0.5, 0_{1\times 9})^\top$ Goal State: $x_\mathcal{G} = (6.0, 0, 3.5, 0_{1\times 9})^\top$

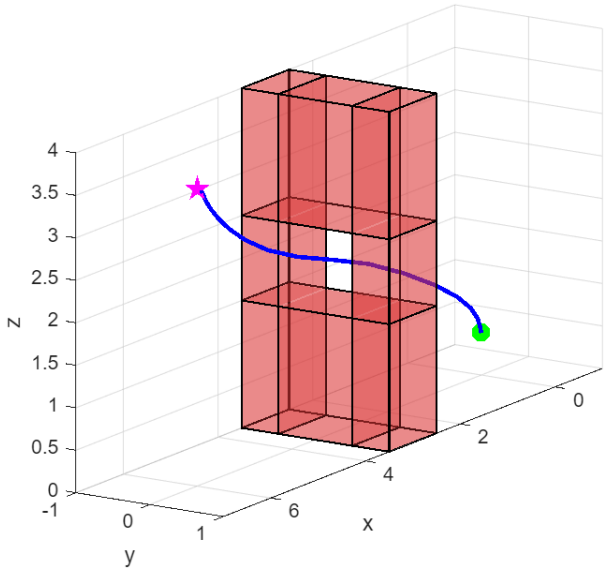
Workspace Bounds: $[-1.0, -1.0, 0.0]^\top \leq p \leq [7.0, 1.0, 4.0]^\top$

Obstacle Constraints:

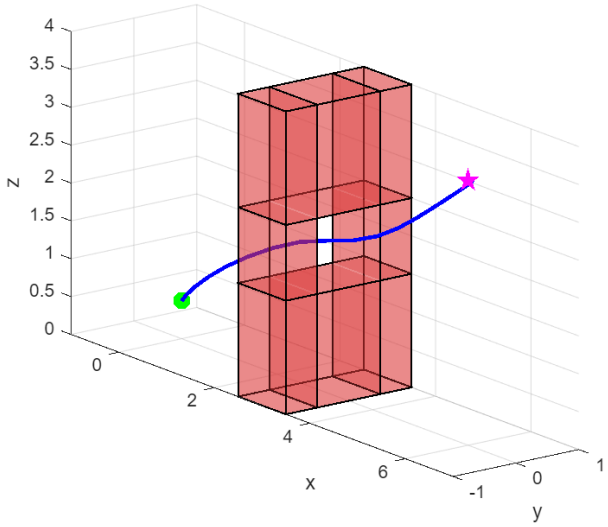
1. $x \in [2.5, 3.5], \quad y \in [-1.0, 1.0], \quad z \in [0.0, 1.5]$
2. $x \in [2.5, 3.5], \quad y \in [-1.0, 1.0], \quad z \in [2.5, 4.0]$
3. $x \in [2.5, 3.5], \quad y \in [-1.0, -0.5], \quad z \in [0.0, 4.0]$
4. $x \in [2.5, 3.5], \quad y \in [0.5, 1.0], \quad z \in [0.0, 4.0]$



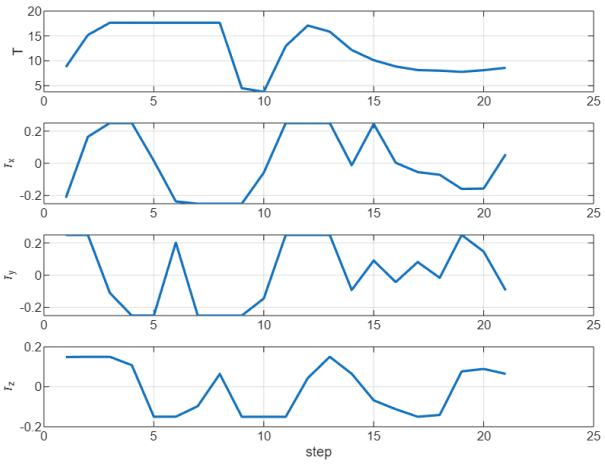
(a) Case 3 trajectory.



(b) Case 3 trajectory.



(c) Case 3 trajectory.



(d) Case 3 trajectory.

Figure 3: Figures a-c show a rendering of the quadrotor trajectory through the Case 3 environment. The initial pose is represented using a green dot, the goal pose is shown as a pink star, the trajectory of the quadrotor is given as a blue line, and the polyhedral obstacles are rendered as translucent red boxes. Figure d shows the evolution of the control inputs over time.

Across all three scenarios, our controller generated dynamically feasible trajectories that reached the goal while satisfying actuator, workspace, and obstacle-avoidance constraints. No collisions were observed despite the stochastic dynamics of the true system.

These experiments demonstrate that local covariance propagation combined with signed-distance tightening can provide scalable stochastic obstacle avoidance for nonlinear quadrotor MPC without expensive particle-based chance-constraint evaluation.

5 Conclusion

This paper presents a chance-constrained nonlinear model predictive control framework for navigation in cluttered environments under stochastic dynamics. Our method combines nonlinear quadrotor dynamics with signed distance field obstacle representations and covariance-based constraint tightening to approximate stochastic collision avoidance in a computationally efficient manner. By projecting predicted positional uncertainty onto local obstacle normal directions, probabilistic safety constraints are transformed into deterministic margins that can be integrated within a receding-horizon NMPC solver. This avoids the computational burden of particle-based chance-constraint methods while retaining sensitivity to local obstacle geometry. Our simulation results of a quadrotor system across multiple 3D navigation scenarios demonstrate that our method can generate dynamically feasible trajectories satisfying actuator, workspace, and obstacle-avoidance constraints under Gaussian process noise. While the present formulation relies on local linearization and Gaussian uncertainty assumptions, the results indicate that covariance-aware SDF tightening is a practical and scalable tool for stochastic aerial navigation. Future work should include recursive feasibility analysis, real-time hardware implementation, moving obstacles, state-estimation uncertainty, and non-Gaussian or learning-based uncertainty models.

References

[1] Lars Blackmore and Masahiro Ono. *Convex Chance Constrained Predictive Control Without Sampling*. doi: 10.2514/6.2009-5876. URL https://arc.aiaa.org/doi/abs/10.2514/6.2009-5876.

[2] Marcello Farina, Luca Giulioni, and Riccardo Scattolini. Stochastic linear model predictive control with chance constraints – a review. *Journal of Process Control*, 44:53–67, 2016. ISSN 0959-1524. doi: https://doi.org/10.1016/j.jprocont.2016.03.005. URL https://www.sciencedirect.com/science/article/pii/S0959152416300130.

[3] Martin Jacquet, Marvin Harms, and Kostas Alexis. Neural nmpc through signed distance field encoding for collision avoidance. *The International Journal of Robotics Research*, December 2025. ISSN 1741-3176. doi: 10.1177/02783649251401223. URL http://dx.doi.org/10.1177/02783649251401223.

[4] David Leprich, Mario Rosenfelder, Mario Hermle, Jingshan Chen, and Peter Eberhard. Model predictive path-following control for a quadrotor. *Proceedings in Applied Mathematics and Mechanics*, 26(1), January 2026. ISSN 1617-7061. doi: 10.1002/pamm.70056. URL http://dx.doi.org/10.1002/pamm.70056.

[5] David Q Mayne, Maria M Seron, and Rakovi. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005.

[6] Ali Mesbah, Ilya V. Kolmanovsky, and Stefano Di Cairano. Stochastic model predictive control. In *Handbook of Model Predictive Control*, pages 1–25. Springer International Publishing, 2018. doi: 10.1007/978-3-319-77489-3_4.

[7] J.B. Rawlings, D.Q. Mayne, and M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017. ISBN 9780975937730. URL https://books.google.com/books?id=MrJctAEACAAJ.

[8] Georg Schildbach, Lorenzo Fagiano, Christoph Frei, and Manfred Morari. The scenario approach for stochastic model predictive control with bounds on closed-loop constraint violations. In *2014 American Control Conference*, pages 5050–5055. IEEE, 2014.

Appendix

This appendix includes the entirety of the MATLAB code necessary to replicate the presented results.

Note: to run the three cases presented in Section 4, please see line 49 of the provided code. You will notice that in order to run any case, you will need to comment/uncomment the cases. To do this quickly in MATLAB, use '`ctrl`' + '`r`' to bulk comment and '`ctrl`' + '`t`' to bulk uncomment

```
clear; clc; close all;
import casadi.*

% MPC Parameters
Ts   = 0.10;
N    = 20;
Tsim = 80;
nSCP = 2;


nx = 12; % [px py pz vx vy vz phi theta psi wx wy wz]
nu = 4;  % [T tau_x tau_y tau_z]

% Model Parameters
m  = 1.0;
g  = 9.81;
Jx = 0.02;
Jy = 0.02;
Jz = 0.04;
J  = diag([Jx Jy Jz]);

T_min = 0.2*m*g; % min thrust
T_max = 1.8*m*g; % max thrust
tau_max = [0.25; 0.25; 0.15]; % torque magnitude limits

angle_max = deg2rad(35); % Euler-angle limits

r_robot = 0.25; % quadrotor radius

% gaussian noise
sigma_acc = [0.25; 0.25; 0.18];
Wacc = diag(sigma_acc.^2);

alpha = 0.05;
beta_cc = gauss_quantile(1 - alpha);

Q  = diag([30 30 35, 4 4 5, 2 2 2, 0.2 0.2 0.2]);
Qf = diag([80 80 100, 8 8 10, 4 4 4, 0.5 0.5 0.5]);
R  = diag([0.08, 0.5, 0.5, 0.5]);

u_hover = [m*g; 0; 0; 0]; % nominal eqbrm input

K_tube = lqr_gain(Ts, m, g, J, Q, R);

D = zeros(nx,3);
D(4:6,:) = eye(3);

```

```
% Box obstacles: (per row) [xmin xmax ymin ymax zmin zmax]
% To run each case, comment/uncomment the others

% Case 1:

x0 = zeros(nx,1);
x0(1:3) = [0; 0; 1.0];

x_goal = zeros(nx,1);
x_goal(1:3) = [8.5; 8.0; 2.0];

pos_min = [-1; -1; 0.2];
pos_max = [10; 10; 4.0];

boxes = [
    2.5 4.0   2.0 3.5   0.5 4.0;
    5.6 7.0   3.4 4.8   0.5 2.6;
    3.2 4.8   6.1 7.6   0.7 3.0
];

% Case 2:

% x0 = zeros(nx,1);
% x0(1:3) = [0; 0; 2.0];
%
% x_goal = zeros(nx,1);
% x_goal(1:3) = [9.0; 0.0; 2.0];
%
% pos_min = [-1; -1; 0.0];
% pos_max = [10.0; 1; 4.0];
%
% boxes = [
%     2.0 3.5   -1.0 1.0   0.0 2.0;
%     5.6 7.0   -1.0 1.0   2.0 4.0
% ];

% Case 3:

% x0 = zeros(nx,1);
% x0(1:3) = [0; 0; 0.5];
%
% x_goal = zeros(nx,1);
% x_goal(1:3) = [6.0; 0.0; 3.5];
%
% pos_min = [-1; -1; 0.0];
% pos_max = [7.0; 1; 4.0];
%
% boxes = [
%     2.5 3.5   -1.0 1.0   0.0 1.5;
%     2.5 3.5   -1.0 1.0   2.5 4.0;
%     2.5 3.5   -1.0 -0.5   0.0 4.0;
%     2.5 3.5   0.5 1.0   0.0 4.0
% ];

```

```
nObs = size(boxes,1);

obs_eps = 1e-3;

rng(2);

% CasADi setup
x_sym = SX.sym('x', nx);
u_sym = SX.sym('u', nu);

xdot_sym = quad_dynamics_euler_sx(x_sym, u_sym, m, g, J);
f_ode = Function('f_ode', {x_sym, u_sym}, {xdot_sym});

k1 = f_ode(x_sym, u_sym);
k2 = f_ode(x_sym + Ts/2*k1, u_sym);
k3 = f_ode(x_sym + Ts/2*k2, u_sym);
k4 = f_ode(x_sym + Ts*k3, u_sym);

x_next_sym = x_sym + Ts/6*(k1 + 2*k2 + 2*k3 + k4);

f_disc = Function('f_disc', {x_sym, u_sym}, {x_next_sym});

A_sym = jacobian(x_next_sym, x_sym);
B_sym = jacobian(x_next_sym, u_sym);

f_A = Function('f_A', {x_sym, u_sym}, {A_sym});
f_B = Function('f_B', {x_sym, u_sym}, {B_sym});

% initilize solver
x_actual = x0;

X_hist = zeros(nx,Tsim+1);
U_hist = zeros(nu,Tsim);
X_hist(:,1) = x_actual;

X_guess = zeros(nx,N+1);
U_guess = repmat(u_hover,1,N);

for k = 1:N+1
    a = (k-1)/N;
    X_guess(:,k) = (1-a)*x0 + a*x_goal;
end

% Simulate and solve
for t = 1:Tsim

    x_nom0 = x_actual;

    for scp = 1:nSCP

        Sigma_seq = propagate_covariance( ...
            f_A, f_B, X_guess, U_guess, K_tube, D, Wacc, N);

        [X_sol, U_sol, success] = solve_quadrotor_nmpc_box_sdf( ...
```

```
            f_disc, x_nom0, x_goal, X_guess, U_guess, ...
            Sigma_seq, boxes, r_robot, beta_cc, obs_eps, ...
            Q, Qf, R, u_hover, ...
            T_min, T_max, tau_max, pos_min, pos_max, angle_max, ...
            N, nx, nu);

        if ~success
            warning('NMPC failed at sim step %d, SCP iter %d.', t, scp);
            X_sol = X_guess;
            U_sol = U_guess;
            break;
        end

        X_guess = X_sol;
        U_guess = U_sol;
    end

    u_nom = U_sol(:,1);

    e = x_actual - X_sol(:,1);
    e(7:9) = wrapToPi(e(7:9));

    u_applied = u_nom + K_tube*e;

    u_applied(1) = min(max(u_applied(1), T_min), T_max);
    u_applied(2:4) = min(max(u_applied(2:4), -tau_max), tau_max);

    x_next = full(f_disc(x_actual, u_applied));

    w_acc = chol(Wacc,'lower') * randn(3,1);
    x_next(4:6) = x_next(4:6) + Ts*w_acc;
    x_next(7:9) = wrapToPi(x_next(7:9));

    X_hist(:,t+1) = x_next;
    U_hist(:,t) = u_applied;

    x_actual = x_next;

    X_guess = [X_sol(:,2:end), X_sol(:,end)];
    U_guess = [U_sol(:,2:end), U_sol(:,end)];

    if norm(x_actual(1:3)-x_goal(1:3)) < 0.25 && norm(x_actual(4:6)) <
       0.25
        X_hist = X_hist(:,1:t+1);
        U_hist = U_hist(:,1:t);
        fprintf('Reached goal at step %d.\n', t);
        break;
    end
end

% DEBUG output, distnace
min_clearance = inf;
for k = 1:size(X_hist,2)
    p = X_hist(1:3,k);
```

```
    for j = 1:nObs
        [d,~] = box_sdf_and_grad(p, boxes(j,:)');
        min_clearance = min(min_clearance, d - r_robot);
    end
end
fprintf('Minimum trajectory-clearance: %.4f m\n', min_clearance);




% Plot
figure('Color','w'); hold on; grid on; axis equal;
xlabel('x'); ylabel('y'); zlabel('z');

plot3(X_hist(1,:), X_hist(2,:), X_hist(3,:), 'b-', 'LineWidth', 2.2);
plot3(x0(1), x0(2), x0(3), 'go', 'MarkerFaceColor','g', 'MarkerSize', 9);
plot3(x_goal(1), x_goal(2), x_goal(3), 'mp', 'MarkerFaceColor','m', '
    MarkerSize', 12);

for j = 1:nObs
    plot_box(boxes(j,:), [0.85 0.2 0.2], 0.35);
end

xlim([pos_min(1), pos_max(1)]);
ylim([pos_min(2), pos_max(2)]);
zlim([pos_min(3), pos_max(3)]);
view(35,25);

figure('Color','w');
subplot(4,1,1); plot(U_hist(1,:), 'LineWidth',1.5); grid on; ylabel('T');
subplot(4,1,2); plot(U_hist(2,:), 'LineWidth',1.5); grid on; ylabel('\
    tau_x');
subplot(4,1,3); plot(U_hist(3,:), 'LineWidth',1.5); grid on; ylabel('\
    tau_y');
subplot(4,1,4); plot(U_hist(4,:), 'LineWidth',1.5); grid on; ylabel('\
    tau_z'); xlabel('step');




% Functions

function [X_sol, U_sol, success] = solve_quadrotor_nmpc_box_sdf( ...
    f_disc, x0, x_goal, X_guess, U_guess, ...
    Sigma_seq, boxes, r_robot, beta_cc, obs_eps, ...
    Q, Qf, R, u_hover, ...
    T_min, T_max, tau_max, pos_min, pos_max, angle_max, ...
    N, nx, nu)

    import casadi.*

    X = SX.sym('X', nx, N+1);
    U = SX.sym('U', nu, N);

```

```
    obj = 0;
    gcon = {};
    lbg = [];
    ubg = [];

    gcon{end+1} = X(:,1) - x0;
    lbg = [lbg; zeros(nx,1)];
    ubg = [ubg; zeros(nx,1)];

    for k = 1:N

        xk = X(:,k);
        uk = U(:,k);

        gcon{end+1} = X(:,k+1) - f_disc(xk, uk);
        lbg = [lbg; zeros(nx,1)];
        ubg = [ubg; zeros(nx,1)];

        dx = xk - x_goal;
        du = uk - u_hover;
        obj = obj + dx'*Q*dx + du'*R*du;

        gcon{end+1} = xk(1:3);
        lbg = [lbg; pos_min];
        ubg = [ubg; pos_max];

        gcon{end+1} = xk(7:9);
        lbg = [lbg; -angle_max*ones(3,1)];
        ubg = [ubg;  angle_max*ones(3,1)];

        p_ref = X_guess(1:3,k);
        Sig_p = Sigma_seq(1:3,1:3,k);

        for j = 1:size(boxes,1)
            box = boxes(j,:)';

            [d_ref, grad_d] = box_sdf_and_grad(p_ref, box);

            sigma_sdf = sqrt(max(grad_d' * Sig_p * grad_d, 0));

            sdf_lin = d_ref + grad_d'*(xk(1:3) - p_ref);

            gcon{end+1} = sdf_lin;
            lbg = [lbg; r_robot + beta_cc*sigma_sdf + obs_eps];
            ubg = [ubg; inf];
        end
    end

    dxN = X(:,N+1) - x_goal;
    obj = obj + dxN'*Qf*dxN;

    gcon{end+1} = X(1:3,N+1);
    lbg = [lbg; pos_min];
    ubg = [ubg; pos_max];
```

```
313
314     z = [reshape(X, nx*(N+1), 1);
315          reshape(U, nu*N, 1)];
316
317     G_all = vertcat(gcon{:});
318
319     nlp = struct('x', z, 'f', obj, 'g', G_all);
320
321     opts = struct;
322     opts.ipopt.print_level = 0;
323     opts.print_time = false;
324     opts.ipopt.max_iter = 300;
325     opts.ipopt.tol = 1e-4;
326     opts.ipopt.acceptable_tol = 1e-3;
327
328     solver = nlpsol('solver','ipopt',nlp,opts);
329
330     lbz = -inf(size(z));
331     ubz =  inf(size(z));
332
333     offset_U = nx*(N+1);
334
335     for k = 1:N
336         idx = offset_U + (k-1)*nu + (1:nu);
337
338         lbz(idx(1)) = T_min;
339         ubz(idx(1)) = T_max;
340
341         lbz(idx(2:4)) = -tau_max;
342         ubz(idx(2:4)) =  tau_max;
343     end
344
345     z0 = [reshape(X_guess, nx*(N+1), 1);
346           reshape(U_guess, nu*N, 1)];
347
348     try
349         sol = solver('x0', z0, 'lbx', lbz, 'ubx', ubz, 'lbg', lbg, 'ubg',
                ubg);
350         zsol = full(sol.x);
351
352         X_vec = zsol(1:nx*(N+1));
353         U_vec = zsol(nx*(N+1)+1:end);
354
355         X_sol = reshape(X_vec, nx, N+1);
356         U_sol = reshape(U_vec, nu, N);
357
358         success = true;
359     catch
360         X_sol = X_guess;
361         U_sol = U_guess;
362         success = false;
363     end
364 end
365
```

```
function Sigma_seq = propagate_covariance(f_A, f_B, Xref, Uref, K, G, W, N)
    nx = size(Xref,1);
    Sigma_seq = zeros(nx,nx,N+1);
    Sigma_seq(:,:,1) = zeros(nx);

    for k = 1:N
        Ak = full(f_A(Xref(:,k), Uref(:,k)));
        Bk = full(f_B(Xref(:,k), Uref(:,k)));

        Acl = Ak + Bk*K;

        Sigma_seq(:,:,k+1) = Acl*Sigma_seq(:,:,k)*Acl' + G*W*G';
        Sigma_seq(:,:,k+1) = 0.5*(Sigma_seq(:,:,k+1)+Sigma_seq(:,:,k+1)');
    end
end

function [d, grad] = box_sdf_and_grad(p, box)
    p = p(:);

    xmin = box(1); xmax = box(2);
    ymin = box(3); ymax = box(4);
    zmin = box(5); zmax = box(6);

    c = [(xmin+xmax)/2; (ymin+ymax)/2; (zmin+zmax)/2];
    h = [(xmax-xmin)/2; (ymax-ymin)/2; (zmax-zmin)/2];

    q = abs(p - c) - h;

    q_pos = max(q, 0);
    outside_dist = norm(q_pos);
    inside_dist = min(max(q(1), max(q(2), q(3))), 0);

    d = outside_dist + inside_dist;

    if outside_dist > 1e-9
        closest = min(max(p, [xmin; ymin; zmin]), [xmax; ymax; zmax]);
        v = p - closest;
        grad = v / norm(v);
    else
        dist_to_faces = [
            abs(p(1)-xmin);
            abs(xmax-p(1));
            abs(p(2)-ymin);
            abs(ymax-p(2));
            abs(p(3)-zmin);
            abs(zmax-p(3))
        ];

        [~,idx] = min(dist_to_faces);

        switch idx
            case 1
                grad = [-1;0;0];
```

```
            case 2
                grad = [1;0;0];
            case 3
                grad = [0;-1;0];
            case 4
                grad = [0;1;0];
            case 5
                grad = [0;0;-1];
            case 6
                grad = [0;0;1];
        end
    end
end

function xdot = quad_dynamics_euler_sx(x, u, m, g, J)
    import casadi.*

    vx = x(4); vy = x(5); vz = x(6);

    phi   = x(7);
    theta = x(8);
    psi   = x(9);

    wx = x(10);
    wy = x(11);
    wz = x(12);

    T = u(1);
    tau = u(2:4);

    R = Rzyx_sx(phi, theta, psi);

    e3 = [0;0;1];

    p_dot = [vx; vy; vz];
    v_dot = (T/m) * R * e3 - [0;0;g];

    omega = [wx; wy; wz];
    omega_dot = inv(SX(J)) * (tau - cross_sx(omega, SX(J)*omega));

    E = euler_rate_matrix_sx(phi, theta);
    eul_dot = E * omega;

    xdot = [p_dot; v_dot; eul_dot; omega_dot];
end

function R = Rzyx_sx(phi, theta, psi)
    cphi = cos(phi);   sphi = sin(phi);
    cth  = cos(theta); sth  = sin(theta);
    cpsi = cos(psi);   spsi = sin(psi);

    Rz = [cpsi -spsi 0;
          spsi  cpsi 0;
          0     0    1];
```

```
473
474     Ry = [cth 0 sth;
475           0   1 0;
476          -sth 0 cth];
477
478     Rx = [1 0 0;
479           0 cphi -sphi;
480           0 sphi  cphi];
481
482     R = Rz*Ry*Rx;
483 end
484
485 function E = euler_rate_matrix_sx(phi, theta)
486     E = [1, sin(phi)*tan(theta), cos(phi)*tan(theta);
487          0, cos(phi),           -sin(phi);
488          0, sin(phi)/cos(theta), cos(phi)/cos(theta)];
489 end
490
491 function c = cross_sx(a,b)
492     c = [a(2)*b(3)-a(3)*b(2);
493          a(3)*b(1)-a(1)*b(3);
494          a(1)*b(2)-a(2)*b(1)];
495 end
496
497 function K = lqr_gain(Ts, m, g, J, Q, R)
498     nx = 12;
499     nu = 4;
500
501     Ac = zeros(nx,nx);
502     Bc = zeros(nx,nu);
503
504     Ac(1,4) = 1;
505     Ac(2,5) = 1;
506     Ac(3,6) = 1;
507
508     Ac(4,8) = g;
509     Ac(5,7) = -g;
510     Bc(6,1) = 1/m;
511
512     Ac(7,10) = 1;
513     Ac(8,11) = 1;
514     Ac(9,12) = 1;
515
516     Bc(10,2) = 1/J(1,1);
517     Bc(11,3) = 1/J(2,2);
518     Bc(12,4) = 1/J(3,3);
519
520     Ad = eye(nx) + Ts*Ac;
521     Bd = Ts*Bc;
522
523     [Kdlqr,~,~] = dlqr(Ad, Bd, Q, R);
524
525     K = -Kdlqr;
526 end
```

```
527
528 function q = gauss_quantile(p)
529     q = -sqrt(2) * erfcinv(2*p);
530 end
531
532 function y = wrapToPi(x)
533     y = mod(x + pi, 2*pi) - pi;
534 end
535
536 function plot_box(box, color, alpha_val)
537     xmin = box(1); xmax = box(2);
538     ymin = box(3); ymax = box(4);
539     zmin = box(5); zmax = box(6);
540
541     V = [
542         xmin ymin zmin;
543         xmax ymin zmin;
544         xmax ymax zmin;
545         xmin ymax zmin;
546         xmin ymin zmax;
547         xmax ymin zmax;
548         xmax ymax zmax;
549         xmin ymax zmax
550     ];
551
552     F = [
553         1 2 3 4;
554         5 6 7 8;
555         1 2 6 5;
556         2 3 7 6;
557         3 4 8 7;
558         4 1 5 8
559     ];
560
561     patch('Vertices', V, 'Faces', F, ...
562           'FaceColor', color, ...
563           'FaceAlpha', alpha_val, ...
564           'EdgeColor', 'k', ...
565           'LineWidth', 1.0);
566 end
```